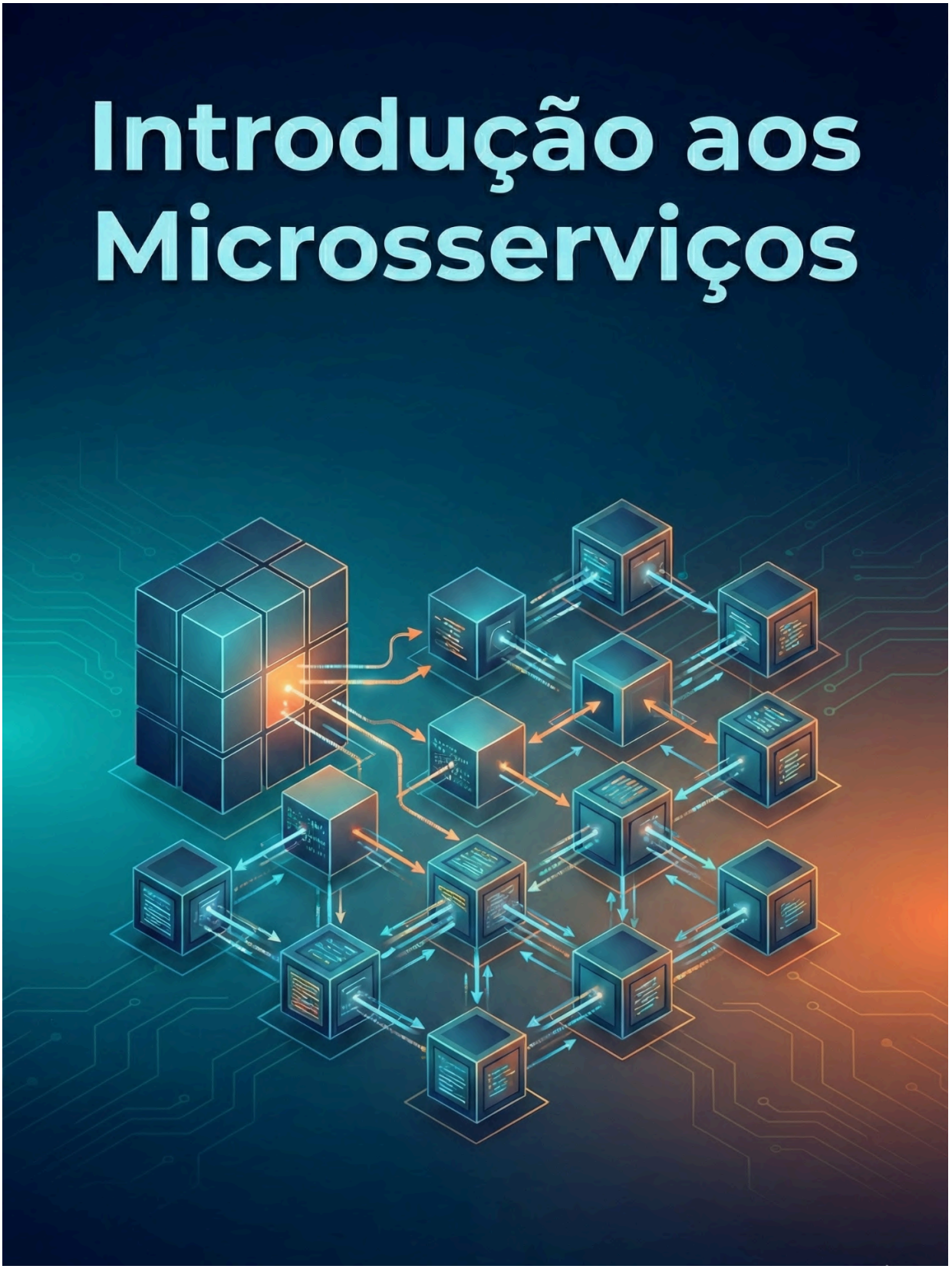
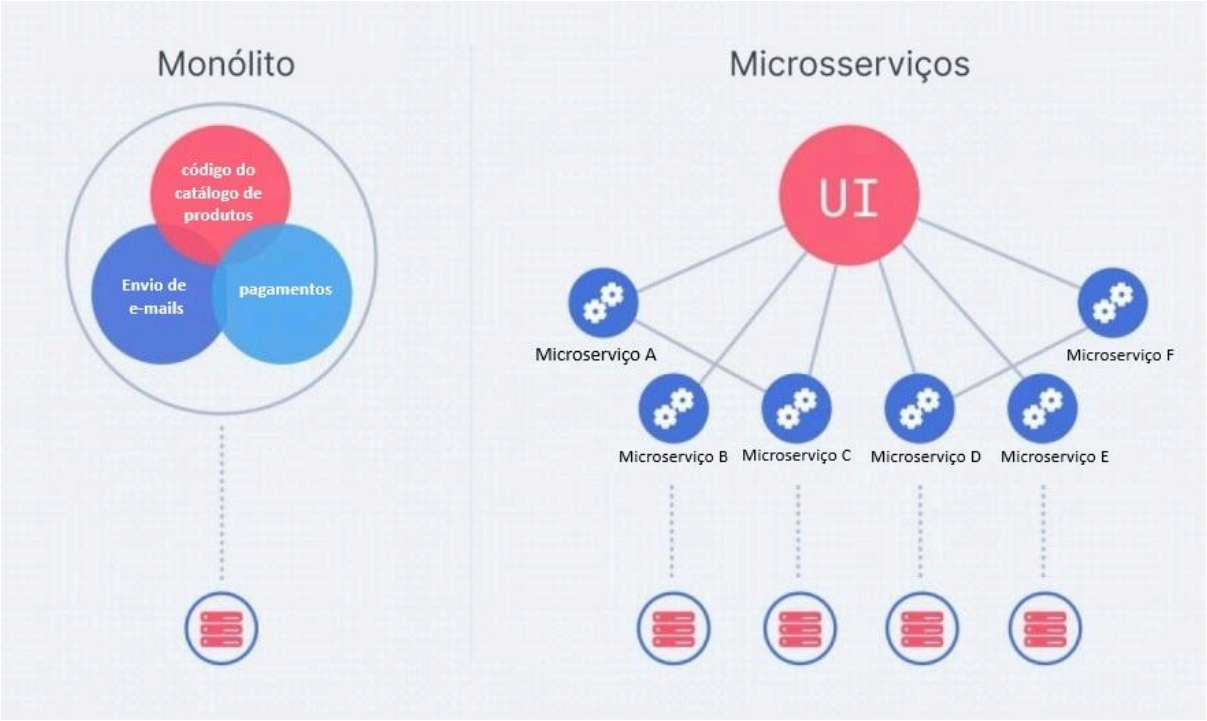
Introdução aos Microsserviços



O Grande Monolito vs. Microserviços (O Time de Especialistas)

Tradicionalmente, aplicações web eram construídas como uma única unidade indivisível, chamada de **Arquitetura Monolítica**. Imagine um "monstro" gigante onde o código do catálogo de produtos, o processamento de pagamentos e o envio de e-mails estão todos misturados no mesmo projeto. Se uma parte quebrar, o monstro inteiro pode cair.

Em contraste, a **Arquitetura de Microsserviços** divide esse sistema em pequenas partes independentes. Cada parte (serviço) é um "especialista" em apenas uma tarefa. Eles trabalham juntos, mas se um falhar, os outros continuam funcionando.



- ***Esquerda (Monolito):*** Um grande bloco/caixa contendo "UI + Regras de Negócio + Acesso a Dados" tudo junto, conectado a um único banco de dados gigante.
- ***Direita (Microsserviços):*** Várias caixas pequenas separadas ("Serviço A", "Serviço B"), cada uma com seu próprio banco de dados pequeno, conectadas por linhas finas (rede).

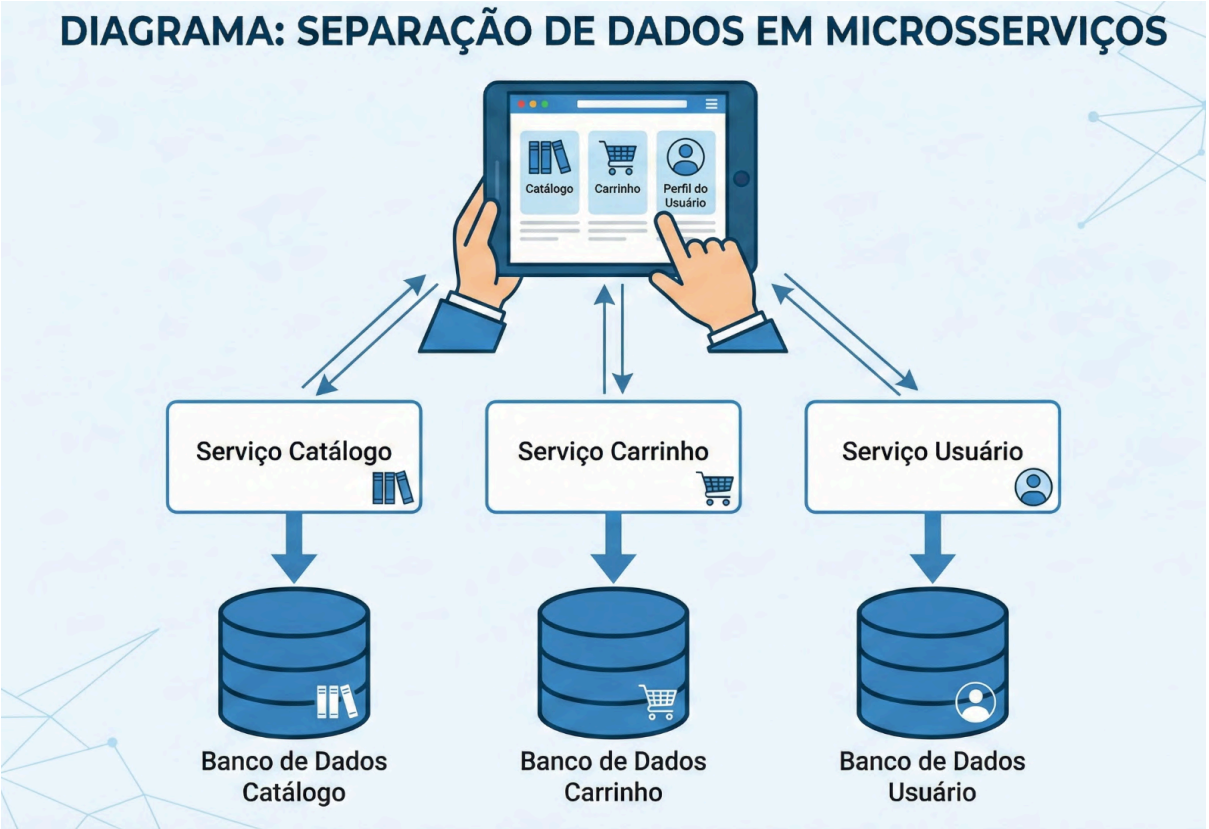
Características Principais

Os microsserviços substituem monólitos, dividindo-os em serviços pequenos, independentes e fracamente acoplados, que se comunicam via APIs (REST ou mensageria). Isso garante escalabilidade e desenvolvimento autônomo (poliglotismo).

Exemplo com um e-commerce:

Todo e-commerce possui esses 3 serviços principais:

1. **Catálogo:** Gerencia produtos (estoque, preço, descrição, busca).
2. **Carrinho:** Controla itens, calcula subtotal e checa disponibilidade.
3. **Usuário:** Gerencia registro, autenticação e dados pessoais.



A imagem ilustra o conceito de **descentralização**, um dos pilares dos microsserviços. Em vez de um "baldão" único de dados (modelo monolito), cada funcionalidade tem sua própria "caixa de ferramentas".

O que está acontecendo na imagem acima:

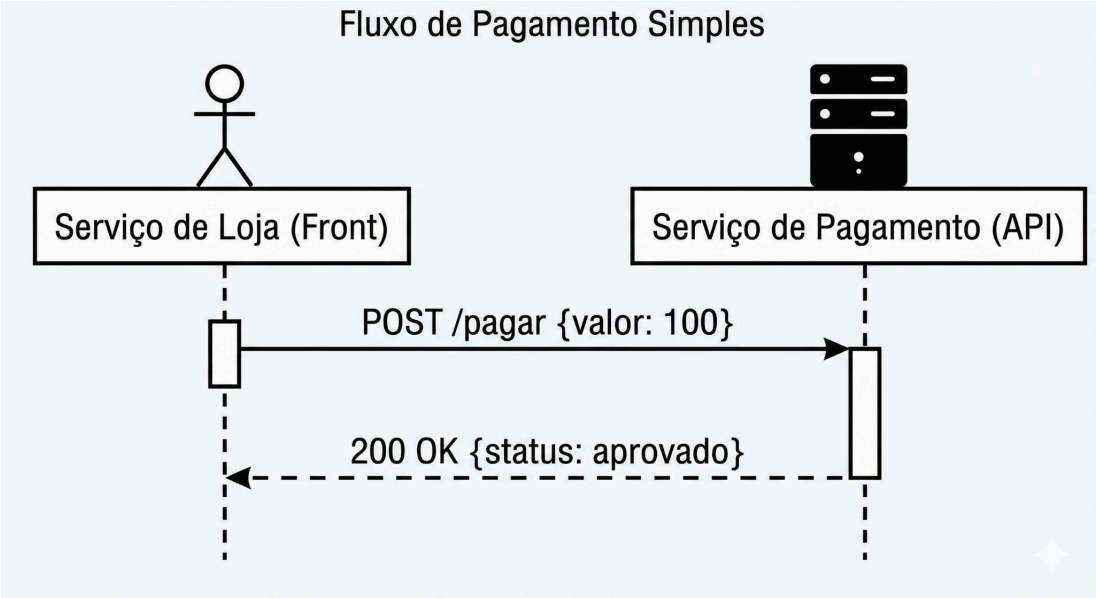
1. A **UI (Interface)** atua como o ponto de entrada, enviando comandos específicos para cada serviço.
2. Cada **microserviço** (Catálogo, Carrinho, Usuário) funciona como uma mini-aplicação independente e isolada.

3. O diferencial é a **autonomia de dados**: cada serviço possui seu próprio banco de dados exclusivo. O serviço de Catálogo não pode ler a tabela de Usuários diretamente; ele precisa "pedir" os dados ao serviço de Usuários.
4. Isso impede o "acoplamento", garantindo que uma falha no banco de usuários não trave o Catálogo.
5. Essa estrutura permite escalar apenas a parte necessária, economizando recursos e evitando gargalos.
6. Na prática, é como uma praça de alimentação: cada quiosque cuida do seu estoque e pedidos.

Como eles conversam? (Comunicação HTTP/REST)

Como os serviços são programas separados (muitas vezes rodando em máquinas diferentes), eles precisam de um "telefone" para se comunicar. O padrão mais comum é o **HTTP/REST**, o mesmo protocolo usado pela Web.

- **Request (Pedido):** O Serviço A envia uma mensagem: *"Ei, Serviço B, faça esse pagamento deste pedido para mim"*. (Geralmente usando JSON).
- **Response (Resposta):** O Serviço B responde: *"Ok, pedido #1023 pago com sucesso"*.



Um diagrama de sequência simples.